Deploying LLMs Locally on Azure with vLLM and TGI

> **Scope:** This guide covers end-to-end deployment of Large Language Models on Azure Virtual Machines using vLLM and Text Generation Inference (TGI). All compute runs entirely within your Azure subscription. No managed AI services (Azure ML, Azure OpenAI) are used.

Table of Contents

Prerequisites

Before starting, ensure you have the following:

- An active Azure subscription with GPU quota approved (request via **Subscriptions → Usage + quotas**)
- Azure CLI installed locally: `curl -sL https://aka.ms/InstallAzureCLIDeb | sudo bash`
- A Hugging Face account with an access token (required for gated models like Llama)
- SSH key pair for VM access
- Docker knowledge (basic)

Login and set defaults

```bash
```

```
az login
az account set --subscription "<YOUR_SUBSCRIPTION_ID>"

# Set variables used throughout this guide
export RESOURCE_GROUP="rg-llm-inference"
export LOCATION="eastus"
export ADMIN_USER="azureuser"
export SSH_KEY_PATH="~/.ssh/id_rsa.pub"
export HF_TOKEN="hf_xxxxxxxxxxxxxxxxxxxx"   # Your Hugging Face token
```

VM Selection

Recommended GPU VM SKUs

SKU	GPU	VRAM	vCPUs	RAM	Best For	Typical Regions
`Standard_NC24ads_A100_v4`	1× A100 80 GB	80 GB	24	220 GB	7B–13B models, single-GPU	eastus, westeurope
`Standard_ND96asr_v4`	8× A100 40 GB	320 GB	96	900 GB	70B models, tensor parallel	eastus, westus2
`Standard_ND96amsr_A100_v4`	8× A100 80 GB	640 GB	96	1900 GB	70B+ quantized or full FP16	eastus, westeurope
`Standard_ND96isr_H100_v5`	8× H100 80 GB	640 GB	96	1900 GB	Highest throughput workloads	eastus2, westus3
`Standard_NC6s_v3`	1× V100 16 GB	16 GB	6	112 GB	7B quantized (budget option)	eastus, westeurope

> **For this guide**, we use `Standard_NC24ads_A100_v4` (1× A100 80 GB) — a good balance of cost and capability for Llama-3.1-8B-Instruct.

Check GPU quota availability

bash

```
az vm list-skus \
  --location $LOCATION \
  --size Standard_NC \
  --output table

# Check your current quota
az vm list-usage \
  --location $LOCATION \
  --output table | grep -i "NC"
```

Option A — vLLM Deployment

vLLM is a high-throughput inference engine built around PagedAttention, offering excellent continuous batching and OpenAI-compatible APIs.

A1. Create Resource Group and Network

bash

```
# Create resource group
az group create \
  --name $RESOURCE_GROUP \
  --location $LOCATION

# Create VNet and subnet
az network vnet create \
  --resource-group $RESOURCE_GROUP \
  --name vnet-llm \
  --address-prefix 10.0.0.0/16 \
  --subnet-name subnet-inference \
  --subnet-prefix 10.0.1.0/24

# Create Network Security Group
az network nsg create \
  --resource-group $RESOURCE_GROUP \
  --name nsg-llm-inference
```

A2. Deploy the VM via Azure CLI

bash

```
az vm create \
  --resource-group $RESOURCE_GROUP \
  --name vm-vllm-inference \
  --image Canonical:ubuntu-24_04-lts:server:latest \
  --size Standard_NC24ads_A100_v4 \
  --admin-username $ADMIN_USER \
  --ssh-key-values $SSH_KEY_PATH \
  --vnet-name vnet-llm \
  --subnet subnet-inference \
  --nsg nsg-llm-inference \
  --public-ip-sku Standard \
  --os-disk-size-gb 256 \
  --data-disk-sizes-gb 512 \
  --storage-sku Premium_LRS \
  --output json
```

> **Tip:** Add `--priority Spot --eviction-policy Deallocate --max-price -1` for up to 90% cost savings on non-critical workloads. See Optimization Strategies.

Capture the public IP:

bash

```
export VM_IP=$(az vm show \
  --resource-group $RESOURCE_GROUP \
  --name vm-vllm-inference \
  --show-details \
  --query publicIps \
  --output tsv)

echo "VM Public IP: $VM_IP"
```

A3. Connect and Prepare the VM

bash

```
ssh $ADMIN_USER@$VM_IP
```

All subsequent commands run **on the VM** unless otherwise noted.

Mount and format the data disk

bash

```
sudo fdisk -l  # Identify the data disk, typically /dev/sdc

sudo parted /dev/sdc --script mklabel gpt
sudo parted /dev/sdc --script mkpart primary 0% 100%
sudo mkfs.ext4 /dev/sdc1
sudo mkdir -p /data/models
echo '/dev/sdc1 /data/models ext4 defaults,nofail 0 2' | sudo tee -a /etc/fstab
sudo mount -a
df -h /data/models
```

A4. Install NVIDIA Drivers

bash

```
sudo apt-get update && sudo apt-get upgrade -y
sudo apt-get install -y linux-headers-$(uname -r) build-essential

# Add NVIDIA package repository
wget https://developer.download.nvidia.com/compute/cuda/repos/ubuntu2404/x86_64/cuda
sudo dpkg -i cuda-keyring_1.1-1_all.deb
sudo apt-get update

# Install CUDA toolkit and drivers
sudo apt-get install -y cuda-toolkit-12-4 nvidia-driver-550-server

sudo reboot
```

After reboot, reconnect and verify:

bash

```
nvidia-smi
```

Expected output excerpt:

```
+-----------------------------------------------------------------------------+
| NVIDIA-SMI 550.xx    Driver Version: 550.xx    CUDA Version: 12.4           |
|-------------------------------+----------------------+----------------------+
| GPU  Name        Persistence-M| Bus-Id        Disp.A | Volatile Uncorr. ECC |
|   0  A100-SXM4-80GB       Off | 00000000:00:00.0 Off |                    0 |
+-----------------------------------------------------------------------------+
```

A5. Install Docker and NVIDIA Container Toolkit

bash

```
# Install Docker
curl -fsSL https://get.docker.com | sudo bash
sudo usermod -aG docker $USER
newgrp docker

# Install NVIDIA Container Toolkit
curl -fsSL https://nvidia.github.io/libnvidia-container/gpgkey | \
  sudo gpg --dearmor -o /usr/share/keyrings/nvidia-container-toolkit-keyring.gpg

curl -s -L https://nvidia.github.io/libnvidia-container/stable/deb/nvidia-container-
  sed 's#deb https://#deb [signed-by=/usr/share/keyrings/nvidia-container-toolkit-ke
  sudo tee /etc/apt/sources.list.d/nvidia-container-toolkit.list

sudo apt-get update
sudo apt-get install -y nvidia-container-toolkit
sudo nvidia-ctk runtime configure --runtime=docker
sudo systemctl restart docker
```

A6. Verify GPU Passthrough

bash

```
# Confirm Docker can see the GPU
docker run --rm --gpus all nvidia/cuda:12.4.0-base-ubuntu22.04 nvidia-smi
```

You should see the same `nvidia-smi` table from inside the container. If you see any error about `no CUDA-capable device`, recheck the driver installation and reboot.

A7. Deploy vLLM — Llama-3.1-8B-Instruct

bash

```
# Pull the official vLLM image
docker pull vllm/vllm-openai:latest

# Create a directory for model cache
mkdir -p /data/models/hf-cache

# Launch the inference server
docker run -d \
  --name vllm-server \
  --gpus all \
  --restart unless-stopped \
  -p 8000:8000 \
  -v /data/models/hf-cache:/root/.cache/huggingface \
  -e HUGGING_FACE_HUB_TOKEN=$HF_TOKEN \
  vllm/vllm-openai:latest \
    --model meta-llama/Llama-3.1-8B-Instruct \
    --dtype bfloat16 \
    --tensor-parallel-size 1 \
    --gpu-memory-utilization 0.92 \
    --max-model-len 8192 \
    --max-num-seqs 256 \
    --enable-chunked-prefill \
    --quantization None \
    --host 0.0.0.0 \
    --port 8000 \
    --served-model-name llama-3.1-8b-instruct
```

Key vLLM launch flags explained

Flag	Value	Description
`--dtype`	`bfloat16`	Native A100 format; use `float16` for V100
`--tensor-parallel-size`	`1`	Set to number of GPUs for multi-GPU (e.g., `8` for ND96)
`--gpu-memory-utilization`	`0.92`	KV cache pool — lower if OOM errors occur
`--max-model-len`	`8192`	Maximum context window (tokens)
`--max-num-seqs`	`256`	Maximum concurrent requests in flight
`--enable-chunked-prefill`	*(flag)*	Interleaves prefill with decode for lower latency
`--quantization`	`None`	Use `awq`, `gptq`, or `fp8` for quantized models

Monitor startup logs

```bash
docker logs -f vllm-server
# Wait for: "INFO:     Application startup complete."
```

Option B — TGI Deployment

Text Generation Inference (TGI) by Hugging Face offers high throughput with continuous batching, flash attention, and native quantization support.

B1. Create Resource Group and Network

> If you already created the resource group and VNet in Option A, skip to B2.

```bash
```

```
az group create \
  --name $RESOURCE_GROUP \
  --location $LOCATION

az network vnet create \
  --resource-group $RESOURCE_GROUP \
  --name vnet-llm \
  --address-prefix 10.0.0.0/16 \
  --subnet-name subnet-inference \
  --subnet-prefix 10.0.1.0/24

az network nsg create \
  --resource-group $RESOURCE_GROUP \
  --name nsg-llm-inference
```

B2. Deploy the VM via Azure CLI

bash

```
az vm create \
  --resource-group $RESOURCE_GROUP \
  --name vm-tgi-inference \
  --image Canonical:ubuntu-24_04-lts:server:latest \
  --size Standard_NC24ads_A100_v4 \
  --admin-username $ADMIN_USER \
  --ssh-key-values $SSH_KEY_PATH \
  --vnet-name vnet-llm \
  --subnet subnet-inference \
  --nsg nsg-llm-inference \
  --public-ip-sku Standard \
  --os-disk-size-gb 256 \
  --data-disk-sizes-gb 512 \
  --storage-sku Premium_LRS \
  --output json

export VM_IP=$(az vm show \
  --resource-group $RESOURCE_GROUP \
  --name vm-tgi-inference \
  --show-details \
  --query publicIps \
  --output tsv)
```

B3. VM Setup — Drivers, Docker, NVIDIA Container Toolkit

The setup steps are identical to vLLM (A3–A6). Run all commands from sections A3 through A6 on this VM.

B4. Deploy TGI — Llama-3.1-8B-Instruct

```bash
# Pull the official TGI image
docker pull ghcr.io/huggingface/text-generation-inference:2.3.1

# Create model cache directory
mkdir -p /data/models/hf-cache

# Launch the TGI inference server
docker run -d \
  --name tgi-server \
  --gpus all \
  --restart unless-stopped \
  -p 8080:80 \
  -v /data/models/hf-cache:/data \
  -e HUGGING_FACE_HUB_TOKEN=$HF_TOKEN \
  ghcr.io/huggingface/text-generation-inference:2.3.1 \
    --model-id meta-llama/Llama-3.1-8B-Instruct \
    --dtype bfloat16 \
    --num-shard 1 \
    --max-input-length 4096 \
    --max-total-tokens 8192 \
    --max-batch-prefill-tokens 16384 \
    --max-concurrent-requests 256 \
    --quantize None \
    --hostname 0.0.0.0 \
    --port 80
```

Key TGI launch flags explained

Flag	Value	Description
`--dtype`	`bfloat16`	Compute precision; use `float16` for older GPUs
`--num-shard`	`1`	Number of GPU shards for tensor parallelism
`--max-input-length`	`4096`	Maximum prompt length in tokens
`--max-total-tokens`	`8192`	`max-input-length` + max new tokens
`--max-batch-prefill-tokens`	`16384`	Controls prefill batching capacity
`--max-concurrent-requests`	`256`	Maximum queued + active requests
`--quantize`	`None`	Use `bitsandbytes`, `awq`, `gptq`, or `eetq`

Monitor TGI startup

```bash
docker logs -f tgi-server
# Wait for: "Connected" and "Ready to serve" messages
```

Optimization Strategies

Performance Optimizations

Quantization — reduce VRAM footprint

For vLLM:

```bash
# AWQ quantization (recommended — best quality/speed tradeoff)
--quantization awq
# Use a pre-quantized model: e.g., TheBloke/Llama-3.1-8B-Instruct-AWQ

# FP8 (A100/H100 native, minimal quality loss)
--quantization fp8
```

For TGI:

```bash
```

```
# bitsandbytes 4-bit NF4
--quantize bitsandbytes-nf4

# AWQ
--quantize awq
```

Tensor parallelism — scale across multiple GPUs

For 70B models on 8× A100:

bash

```
# vLLM
--tensor-parallel-size 8

# TGI
--num-shard 8
```

Flash Attention 2

Both vLLM and TGI enable Flash Attention 2 automatically on compatible hardware (Ampere and above). No additional flags are needed.

Continuous batching

vLLM uses continuous batching by default via the `--enable-chunked-prefill` flag. TGI uses continuous batching natively. Both avoid padding waste inherent in static batching.

Cost Optimizations

Spot instances (up to 90% savings)

bash

```
az vm create \
  --resource-group $RESOURCE_GROUP \
  --name vm-llm-spot \
  --priority Spot \
  --eviction-policy Deallocate \
  --max-price -1 \
  # ... (rest of vm create args)
```

> **Note:** Use spot only for stateless, fault-tolerant inference. Configure auto-restart via Azure VM Scale Sets or an Azure Function that monitors and re-deploys on eviction.

Reserved instances

For production workloads running >6 months, Azure Reserved VM Instances offer 30–60% savings over pay-as-you-go. Purchase via **Azure Portal → Reservations**.

Auto-shutdown for development VMs

```bash
az vm auto-shutdown \
  --resource-group $RESOURCE_GROUP \
  --name vm-vllm-inference \
  --time 2200 \
  --email "your@email.com"
```

Right-size for your model

Model Size	VRAM Needed (BF16)	Recommended SKU
7B–8B	~16 GB	NC24ads_A100_v4 (80 GB — use quantization for smaller VMs)
13B	~28 GB	NC24ads_A100_v4
34B	~70 GB	NC24ads_A100_v4 (AWQ) or ND96asr_v4
70B	~140 GB	ND96asr_v4 (2× A100 40 GB)
70B FP16	~140 GB	ND96amsr_A100_v4 (2× A100 80 GB)

Security & Networking

NSG Rules — Principle of Least Privilege

By default, block all inbound traffic and only open what is needed.

```bash
```

```
# Block all inbound (deny-all rule at low priority)
az network nsg rule create \
  --resource-group $RESOURCE_GROUP \
  --nsg-name nsg-llm-inference \
  --name deny-all-inbound \
  --priority 4096 \
  --direction Inbound \
  --access Deny \
  --protocol "*" \
  --source-address-prefixes "*" \
  --destination-port-ranges "*"

# Allow SSH only from your corporate IP range
az network nsg rule create \
  --resource-group $RESOURCE_GROUP \
  --nsg-name nsg-llm-inference \
  --name allow-ssh-corp \
  --priority 100 \
  --direction Inbound \
  --access Allow \
  --protocol Tcp \
  --source-address-prefixes "<YOUR_CORP_IP_CIDR>" \
  --destination-port-ranges 22

# Allow inference port from internal VNet only (NOT public internet)
az network nsg rule create \
  --resource-group $RESOURCE_GROUP \
  --nsg-name nsg-llm-inference \
  --name allow-inference-internal \
  --priority 200 \
  --direction Inbound \
  --access Allow \
  --protocol Tcp \
  --source-address-prefixes "10.0.0.0/16" \
  --destination-port-ranges 8000 8080
```

> **Security principle:** Never expose port 8000 or 8080 directly to the public internet. Route all API traffic through an Application Gateway or Azure API Management with authentication.

Private Endpoints and VNet Peering

For production, connect consumer services (App Service, AKS, Functions) to the inference VM through VNet peering, keeping all traffic on the Azure backbone:

bash

```
# Peer your application VNet to the inference VNet
az network vnet peering create \
  --name peering-app-to-inference \
  --resource-group $RESOURCE_GROUP \
  --vnet-name vnet-app \
  --remote-vnet vnet-llm \
  --allow-vnet-access

az network vnet peering create \
  --name peering-inference-to-app \
  --resource-group $RESOURCE_GROUP \
  --vnet-name vnet-llm \
  --remote-vnet vnet-app \
  --allow-vnet-access
```

API Authentication with Nginx Reverse Proxy

Never expose vLLM or TGI directly. Place an Nginx proxy in front with token-based authentication:

bash

```
sudo apt-get install -y nginx apache2-utils

# Generate a bearer token
export API_TOKEN=$(openssl rand -hex 32)
echo "API Token: $API_TOKEN"  # Store this securely (Azure Key Vault)

# Create Nginx config
sudo tee /etc/nginx/sites-available/llm-proxy > /dev/null <<'EOF'
server {
    listen 443 ssl;
    server_name _;

    ssl_certificate /etc/ssl/certs/llm.crt;
    ssl_certificate_key /etc/ssl/private/llm.key;
    ssl_protocols TLSv1.2 TLSv1.3;
    ssl_ciphers HIGH:!aNULL:!MD5;

    location / {
        # Token authentication
        if ($http_authorization != "Bearer YOUR_API_TOKEN_HERE") {
            return 401 '{"error": "Unauthorized"}';
        }

        proxy_pass http://127.0.0.1:8000;  # or 8080 for TGI
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_read_timeout 300;
        proxy_send_timeout 300;
    }
}
EOF

# Generate self-signed cert (replace with ACM/Let's Encrypt for production)
sudo openssl req -x509 -nodes -days 365 -newkey rsa:2048 \
  -keyout /etc/ssl/private/llm.key \
  -out /etc/ssl/certs/llm.crt \
  -subj "/CN=llm-inference"

sudo ln -s /etc/nginx/sites-available/llm-proxy /etc/nginx/sites-enabled/
sudo nginx -t && sudo systemctl reload nginx
```

Store Secrets in Azure Key Vault

bash

```
# Create Key Vault
az keyvault create \
  --name kv-llm-secrets \
  --resource-group $RESOURCE_GROUP \
  --location $LOCATION \
  --sku standard

# Store HF token and API token
az keyvault secret set \
  --vault-name kv-llm-secrets \
  --name HuggingFaceToken \
  --value "$HF_TOKEN"

az keyvault secret set \
  --vault-name kv-llm-secrets \
  --name InferenceApiToken \
  --value "$API_TOKEN"

# Assign the VM's managed identity access to the vault
az vm identity assign \
  --resource-group $RESOURCE_GROUP \
  --name vm-vllm-inference

VM_PRINCIPAL=$(az vm show \
  --resource-group $RESOURCE_GROUP \
  --name vm-vllm-inference \
  --query identity.principalId \
  --output tsv)

az keyvault set-policy \
  --name kv-llm-secrets \
  --object-id $VM_PRINCIPAL \
  --secret-permissions get list
```

Validation

Validate vLLM (OpenAI-Compatible API)

Health check

bash

```bash
curl -s http://$VM_IP:8000/health
# Expected: {"status":"ok"}
```

List available models

bash

```bash
curl -s http://$VM_IP:8000/v1/models | python3 -m json.tool
```

Chat completion request

bash

```bash
curl -s http://$VM_IP:8000/v1/chat/completions \
  -H "Content-Type: application/json" \
  -d '{
    "model": "llama-3.1-8b-instruct",
    "messages": [
      {
        "role": "system",
        "content": "You are a helpful assistant."
      },
      {
        "role": "user",
        "content": "Explain the difference between vLLM and TGI in two sentences."
      }
    ],
    "max_tokens": 256,
    "temperature": 0.7,
    "stream": false
  }' | python3 -m json.tool
```

Streaming completion

bash

```
curl -s http://$VM_IP:8000/v1/chat/completions \
  -H "Content-Type: application/json" \
  -d '{
    "model": "llama-3.1-8b-instruct",
    "messages": [{"role": "user", "content": "Write a haiku about cloud computing."}
    "max_tokens": 64,
    "stream": true
  }'
```

Validate TGI

Health check

bash

```
curl -s http://$VM_IP:8080/health
# Expected: HTTP 200 OK
```

Generate text (TGI native API)

bash

```
curl -s http://$VM_IP:8080/generate \
  -H "Content-Type: application/json" \
  -d '{
    "inputs": "<|begin_of_text|><|start_header_id|>user<|end_header_id|>\n\nExplain
    "parameters": {
      "max_new_tokens": 128,
      "temperature": 0.7,
      "top_p": 0.95,
      "do_sample": true
    }
  }' | python3 -m json.tool
```

Streaming with TGI

bash

```
curl -s http://$VM_IP:8080/generate_stream \
  -H "Content-Type: application/json" \
  -d '{
    "inputs": "<|begin_of_text|><|start_header_id|>user<|end_header_id|>\n\nList 3 G
    "parameters": {
      "max_new_tokens": 256,
      "temperature": 0.7
    }
  }'
```

TGI via OpenAI-compatible endpoint

TGI also exposes an OpenAI-compatible API at `/v1/`:

bash

```
curl -s http://$VM_IP:8080/v1/chat/completions \
  -H "Content-Type: application/json" \
  -d '{
    "model": "tgi",
    "messages": [{"role": "user", "content": "Hello, who are you?"}],
    "max_tokens": 128
  }' | python3 -m json.tool
```

Benchmarking throughput

bash

```
# Install vLLM benchmark tool (run on the VM)
pip install vllm

python -m vllm.entrypoints.openai.run_batch \
  --model llama-3.1-8b-instruct \
  --base-url http://localhost:8000/v1 \
  --input-len 512 \
  --output-len 128 \
  --num-prompts 100
```

Container fails to start — GPU not found

```bash
docker logs vllm-server 2>&1 | grep -i "cuda\|gpu\|nvidia"
# Check that nvidia-ctk is configured
sudo nvidia-ctk runtime configure --runtime=docker
sudo systemctl restart docker
```

Out of memory (OOM) during model load

Reduce `--gpu-memory-utilization` (vLLM) or add `--quantize bitsandbytes-nf4` (TGI):

```bash
# vLLM: reduce from 0.92 to 0.85
--gpu-memory-utilization 0.85

# TGI: enable 4-bit quantization
--quantize bitsandbytes-nf4
```

Model download stuck or failing

```bash
# Verify your HF token has read access for gated models
docker exec vllm-server env | grep HUGGING_FACE_HUB_TOKEN

# Pre-download the model before launching the container
docker run --rm \
  -v /data/models/hf-cache:/root/.cache/huggingface \
  -e HUGGING_FACE_HUB_TOKEN=$HF_TOKEN \
  python:3.11-slim \
  pip install huggingface_hub && \
  python -c "from huggingface_hub import snapshot_download; snapshot_download('meta-
```

Slow first request

The first request triggers CUDA kernel compilation (JIT warm-up). Warm up the server after startup:

```bash
```

```
for i in {1..3}; do
  curl -s http://localhost:8000/v1/chat/completions \
    -H "Content-Type: application/json" \
    -d '{"model":"llama-3.1-8b-instruct","messages":[{"role":"user","content":"hi"}]
    > /dev/null
done
echo "Warm-up complete"
```

Quick Reference: vLLM vs TGI

Dimension	vLLM	TGI
API format	OpenAI-compatible (native)	Own API + OpenAI-compatible
Batching	PagedAttention + chunked prefill	Continuous batching
Quantization	AWQ, GPTQ, FP8, bitsandbytes	bitsandbytes, AWQ, GPTQ, EETQ
Multi-GPU	Tensor + pipeline parallel	Tensor parallel (sharding)
LoRA adapters	✅ Dynamic LoRA loading	✅ Static at startup
Streaming	✅ SSE	✅ SSE
Docker image	`vllm/vllm-openai`	`ghcr.io/huggingface/text-generation-inference`
Default port	8000	80 (mapped to 8080 in this guide)
Best use case	High concurrency, OpenAI drop-in	Hugging Face ecosystem integration

---

Documentation version: 1.0 — Tested with vLLM 0.6.x, TGI 2.3.x, CUDA 12.4, Ubuntu 24.04 LTS on Azure NC24ads_A100_v4.